Optimizing Multi-Robot Collaboration using Turtlebot3

CDE2605R (UREx) Report

Name	Student No.
Jansen Ken Pegrasio	A0305755B
Chen Yu Hsin	A0309013U

Supervisor: Nicholas Chew

Acknowledgements

We would like to express our sincere gratitude to our supervisor, Nicholas Chew, for his invaluable guidance and feedback throughout this project. We also thank the NUS College of Design and Engineering for providing the laboratory facilities and equipment, including the TurtleBot3 platforms used in this study. Finally, we are grateful to our peers and teaching assistants in CDE2605R for their support and constructive discussions.

Abstract

This report investigates end-to-end communication latency in a multi-robot system based on TurtleBot3 platforms and ROS 2, with the aim of identifying practical infrastructure configurations for educational settings such as the NUS Innovation and Design Programme (IDP). Two controlled experiments were conducted: the first comparing two network types (a Samsung Galaxy A25 5G mobile hotspot and a Huawei router) under increasing system load, and the second comparing Fast DDS and Cyclone DDS middleware implementations at higher workload intensities including autonomous navigation via Nav2. Results show that the mobile hotspot provides consistently lower latency and fewer message drops than the Huawei router as the number of active robots scales from one to four. Cyclone DDS significantly outperforms Fast DDS at higher workloads due to its multi-threaded processing pipeline, while Fast DDS's synchronous execution model leads to latency degradation and message loss. Based on these findings, a two-robot configuration using a mobile hotspot and Cyclone DDS is recommended as a reliable and practical baseline for multi-robot deployments within the IDP.

Table of Contents

Acknowledgements **2**
Abstract **3**
Table of Contents **4**
List of Figures **5**
1. Introduction **6**
2. Literature Review **7**
2.1 Multi-robot Frameworks 7
2.2 Message Passing 7
2.2.1 ROS 2 Executors 7
2.2.2 Operating System Scheduler 8
2.2.3 Network Application Layer: ROS 2 Middleware 9
2.2.4 Network Transport Layer: UDP versus TCP 10
2.3 DDS Implementation 10
3. Methodology **12**
3.1 Turtlebot3 Setup 12
3.2 Cartographer Setup 13
3.3 Nav2 Setup 14
3.4 Autonomous Navigation Setup 15
3.5 Experimental Overview 15
3.4.1 Experiment 1: Network Infrastructure vs System Load 16
3.4.2 Experiment 2: DDS Middleware vs Workload Intensity 17
4. Results and Discussion **19**
4.1 Experiment 1: Network Infrastructure vs System Load 19
4.1.1 Results 19
4.1.2 Discussion 20
4.2 Experiment 2: DDS Middleware vs Workload Intensity 20
4.2.1 Results 20
4.2.2 Discussion 22
5. Future Improvements **25**
5.1 Migration to ROS 2 Jazzy 25
5.2 Exploration of Zenoh-Based Communication 25
5.3 Multi-Agent SLAM and Collaborative Mapping 25
6. Conclusion **27**
Appendix 1: Project Timeline **28**
Appendix 2: Bill of Materials **29**

List of Figures

Fig. 1. ROS 2 Executor Workflow ……
Fig. 2. Operating System Process State Diagram ……
Fig. 3. ROS Network Application Layer ……
Fig. 4. Turtlebot3 Burger Model …….
Fig. 5. Namespaces in TurtleBot3 Multi-Robot Collaboration ……
Fig. 6. Mobile Hotspot versus Huawei Router Average Latency ……
Fig. 7. Mobile Hotspot versus Huawei Router Message Drops ……
Fig. 8. Fast DDS versus Cyclone DDS Average Latency ……
Fig. 9. Fast DDS versus Cyclone DDS Message Drops ……
Fig. 10. RViz View after 5 Minutes (Nav2 Stack with Robot Bringup) ……
Fig. 11. Two-Robot Autonomous Navigation Maps (Nav2) in Cyclone DDS ……

1. Introduction

Multi-robot systems are increasingly becoming a central paradigm in modern robotics, enabling collaborative task execution, distributed sensing, and scalable autonomy across a wide range of applications such as search and rescue, autonomous navigation, and smart environments. In such systems, multiple robotic agents operate concurrently and exchange information in real time, forming a distributed network of computational and physical entities.

However, the effectiveness of multi-robot collaboration is highly dependent on the underlying communication infrastructure. Reliable and low-latency message exchange is critical for maintaining coordination, ensuring timely decision-making, and preventing inconsistencies in shared system states. As the number of robots increases, the communication load grows, introducing challenges such as network congestion, message delays, and middleware overhead. These factors can significantly degrade system performance if not properly understood and managed.

This challenge is particularly relevant in educational settings such as the NUS Innovation and Design Programme (IDP). Building upon CDE2310, an existing IDP module that introduces students to autonomous single-robot systems, covering sensor integration, motion control, and basic ROS-based navigation, the programme aims to extend its curriculum to multi-robot systems where multiple agents perform distinct tasks concurrently. As the system scales, it becomes essential to evaluate whether the available infrastructure, often constrained by student-accessible hardware and networking resources, can support the increased workload. In this context, a systematic examination of communication performance under realistic constraints is necessary to ensure the reliability and feasibility of such deployments.

This study aims to quantitatively analyze the communication efficiency of a distributed robotic system by measuring end-to-end latency under varying conditions. Specifically, it investigates how student-accessible network infrastructures and different Data Distribution Service (DDS) middleware implementations impact latency as system workload increases. By grounding the experimental design in realistic IDP settings, this work provides practical insights into the performance limitations and trade-offs that arise in multi-robot deployments within educational environments.

Ultimately, the findings of this study contribute to a deeper understanding of how communication infrastructure influences system-level behavior in multi-robot systems, and offer guidance for designing more robust and efficient robotic applications.

2. Literature Review

2.1 Multi-robot Frameworks

There currently exist many open-source multi-robot frameworks which can be leveraged upon to achieve two main subtasks. The creation of a common reference frame for all robots as well as multi-agent pathfinding to coordinate the actions of robots through potential chokepoints like doors and lifts.

ROS 2 (Robot Operating System 2) is a widely used robotics middleware that supports multiple robots through distributed communication, tools, and libraries. It is fully open source and has broad community and industry adoption. Currently Robotis supports teleoperation of multiple robots from a single remote PC using a unique namespace[1].

Whilst this namespace separation allows for control of multiple Turtlebot3s within the environment, they are operating independently and do not collaborate on a global occupancy grid nor coordinate their movements within the environment, requiring further modification for deployment of autonomous multi-robot systems. Additionally, running multiple navigation instances off separate local maps is computationally inefficient on the remote PC as compared to a central navigation algorithm using a global map.

Open RMF is a platform built on top of ROS 2, Open-RMF enables interoperability and coordination between heterogeneous robot fleets and physical infrastructure, facilitating large scale deployments in industry. Open-RMF serves more as a fleet manager[2] to deconflict and schedule movement of a fleet of robots on fixed routes in static environments rather than explore and map dynamic environments. Whilst Open-RMF may not handle our SLAM needs, it could provide a framework for fleet level task coordination and hardware interfacing[3].

Seeing as no single off the shelf platform fulfills the requirements of our project, we would likely be building off ROS2's namespace separation to transmit LIDAR scans from each Turtlebot to the remote machine, build each robot's local occupancy grid map on the remote machine and merge them into a global map which would be used to send navigation commands to each Turtlebot.

2.2 Message Passing

Performance in this study is evaluated using latency. As introduced in Chapter 2.1, the system uses ROS 2 as the multi-robot framework. To better understand the sources of latency, this section of the literature review outlines how messages flow from one end of the system to the other.

2.2.1 ROS 2 Executors

Before a message is transmitted to the receiving node, it is first handled internally by ROS 2. In a typical ROS 2 program, functions such as *spin* or *spin_once* are used to process incoming data. When

---

[1] *Examples*. (2025). ROBOTIS E-Manual. https://emanual.robotis.com/docs/en/platform/turtlebot3/basic_examples/#load-multiple-turtlebot3s

[2] *RMF Core Overview - Programming Multiple Robots with ROS 2*. (2025). Github.io. https://osrf.github.io/ros2multirobotbook/rmf-core.html

[3] *Integration - Programming Multiple Robots with ROS 2*. (2025). Github.io. https://osrf.github.io/ros2multirobotbook/integration.html

spin is called, an Executor instance is created, and the current thread continuously queries the *rcl* (ROS Client Library) and middleware layers for new messages and events, invoking the corresponding callback functions when they are available[4].

On the receiving side, incoming messages are not immediately processed upon arrival. Instead, the ROS documentation stated that they are stored in the middleware (DDS layer) until the Executor retrieves them and executes the associated callback. This means that message processing depends on when the Executor is able to handle the event, as illustrated in Fig. 1.

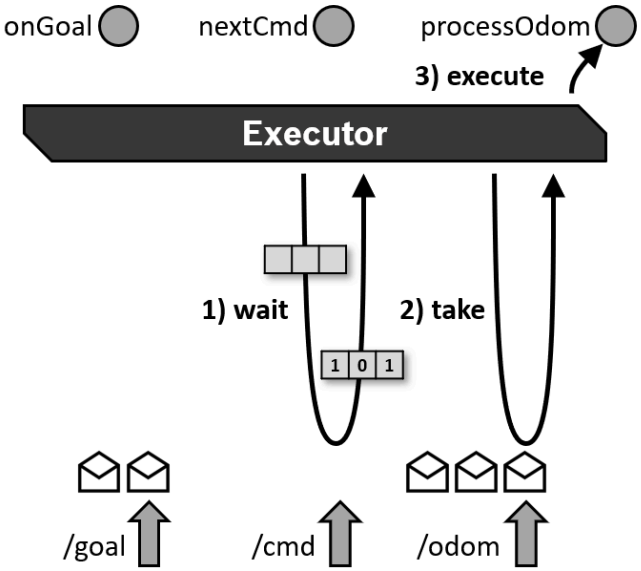


Fig 1. ROS 2 Executor Workflow

As a result, there exists an inherent delay introduced by the Executor. If the Executor is currently busy executing other callbacks or handling multiple events, newly arrived messages must wait before being processed. Therefore, messages are not processed instantly upon arrival, but are instead subject to the scheduling behavior of the Executor.

2.2.2 Operating System Scheduler

From the perspective of the operating system, the ROS 2 Executor runs as a process[5]. A process does not execute continuously; instead, it transitions between states such as running, ready, and waiting, as managed by the OS scheduler and illustrated in Fig. 2[6].

[4] ROS 2 Documentation. (2026). Executors — ROS 2 Concepts. https://docs.ros.org/en/rolling/Concepts/Intermediate/About-Executors.html
[5] Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). Operating System Concepts (10th ed., p. 106). Wiley.
[6] Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). Operating System Concepts (10th ed., p. 107-109). Wiley.

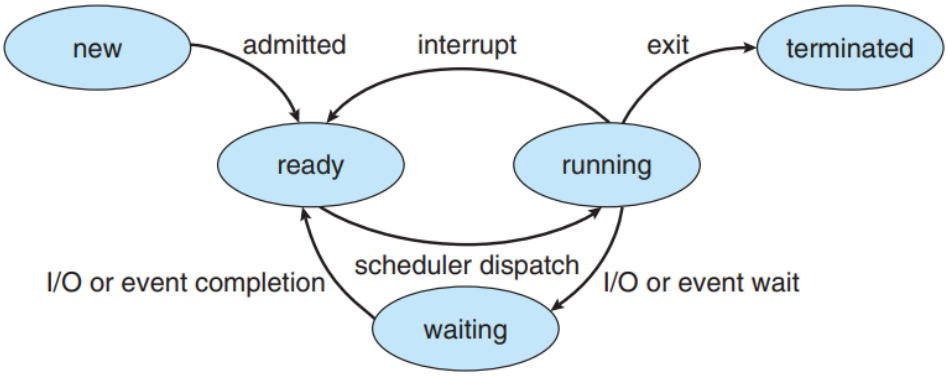


Fig 2. Operating System Process State Diagram

As a result, the Executor may not always run immediately even when messages are available. It can be delayed while waiting in the ready queue, blocked by I/O, or preempted by other processes. In addition, context switching between processes introduces overhead[7].

These factors contribute to latency at the operating system level, meaning that message processing delay is not only caused by ROS 2 and the network, but also by OS scheduling behavior.

2.2.3 Network Application Layer: ROS 2 Middleware

After accounting for delays at the application level, the latency contribution from the network layer will be reviewed. The ROS 2 documentation describes the communication flow across different layers, as illustrated in Fig. 3. This flow resides within the Application Layer of the five-layer Internet protocol stack[8].

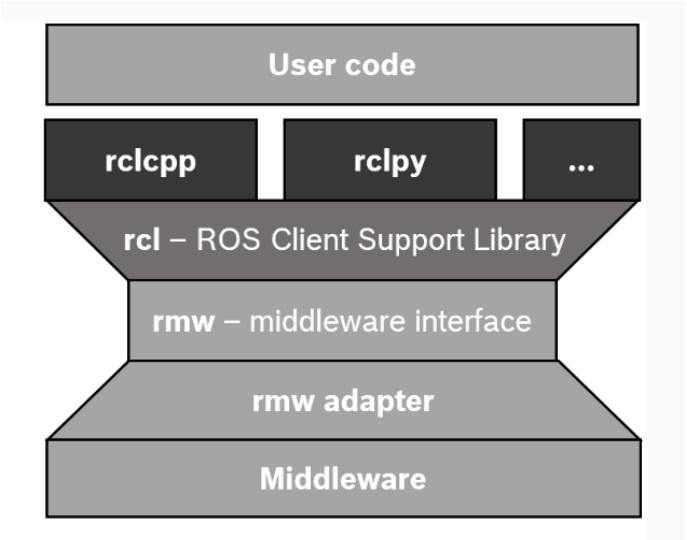


Fig 3. ROS Network Application Layer

[7] Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). Operating System Concepts (10th ed., p. 205-234). Wiley.
[8] Kurose, J. F., & Ross, K. W. (2013). *Computer Networking: A Top-Down Approach* (6th ed., pp. 49–53). Pearson.

As shown in the figure, both the publisher and subscriber first interact with the rcl (ROS Client Library). Messages are then passed through the ROS 2 middleware[9], which is based on Data Distribution Service (DDS). The behaviour of DDS is governed by a set of configurations known as Quality of Service (QoS) policies[10].

QoS parameters in DDS define how messages are handled during communication. One important parameter is Depth, which specifies the size of the message queue[11]. If the incoming message rate exceeds the processing rate and the queue reaches its maximum capacity, some messages may be dropped. Therefore, under high-load conditions, limited queue size can directly lead to message loss.

2.2.4 Network Transport Layer: UDP versus TCP

Different DDS implementations may utilize transport protocols at the transport layer that influence message delivery behaviour. In this study, Fast DDS and Cyclone DDS are considered, and both use UDP as the default transport protocol[12] [13].

UDP is a connectionless protocol, meaning there is no handshaking between the sender and receiver before data transmission. As a result, messages are sent without guarantees of delivery, ordering, or retransmission. If packet loss occurs during transmission, the data is not resent. In contrast, TCP is a connection-oriented protocol that establishes a connection through a handshake mechanism and provides reliability through acknowledgements and retransmissions. Therefore, compared to TCP, the use of UDP introduces the possibility of message loss during transmission[14].

2.3 DDS Implementation

Beyond differences in transport protocols, different DDS frameworks also exhibit variations in their internal implementation.

In Fast DDS, the default configuration uses a synchronous publication mode, where message transmission is performed directly within the application thread. This results in a blocking execution model, in which the publishing operation may delay the application until the message is handled[15].
In contrast, Cyclone DDS employs a multi-threaded processing pipeline, where message reception, queuing, and delivery are handled by separate threads. This allows communication tasks to be distributed across stages and executed concurrently, improving throughput and reducing blocking in the application layer[16].

---

[9] ROS 2 Documentation. (2026). Executors — ROS 2 Concepts. https://docs.ros.org/en/rolling/Concepts/Intermediate/About-Executors.html
[10] Woodall, W. (2019). ROS on DDS. https://design.ros2.org/articles/ros_on_dds.html
[11] MathWorks. (2026). Manage Quality of Service Policies in ROS 2. https://www.mathworks.com/help/ros/ug/manage-quality-of-service-policies-in-ros2.html
[12] eProsima. (2026). *Fast DDS Library Overview*. https://fast-dds.docs.eprosima.com/en/3.x/fastdds/library_overview/library_overview.html
[13] Eclipse Foundation. (2023). *Cyclone DDS Specifics*. https://cyclonedds.io/docs/cyclonedds/0.10.5/config/cyclonedds_specifics.html
[14] Kurose, J. F., & Ross, K. W. (2013). *Computer Networking: A Top-Down Approach* (6th ed., pp. 198-258). Pearson.
[15] ROS 2. (2026). *rmw_fastrtps: ROS Middleware Implementation for eProsima Fast DDS*. https://github.com/ros2/rmw_fastrtps
[16] Eclipse Foundation. (2026). *Cyclone DDS Data Path Configuration*. https://cyclonedds.io/docs/cyclonedds/latest/config/data-path-config.html

3. Methodology

To design a reliable robotic system, it is necessary to quantitatively measure the communication efficiency between system components, especially under varying workload conditions.

In this study, the end-to-end latency is used as the primary performance metric, as it captures the delay experienced by messages transmitted between distributed nodes. As discussed in Chapter 2.2, the measured end-to-end latency in this experiment consists of multiple contributing factors, including network transmission delay (Network layers 1–4), middleware queueing delay, ROS 2 Executor overhead, as well as operating system scheduling and context switching overhead.

3.1 Turtlebot3 Setup

In this research, the TurtleBot3, specifically the burger model, is employed as the primary robotic platform. TurtleBot3 is a modular and customizable mobile robot equipped with key components such as a LiDAR sensor, a Raspberry Pi onboard computer, and an OpenCR control board[17]. Its flexible architecture allows it to be adapted for a wide range of autonomous tasks, making it well-suited for experimentation in multi-robot systems.

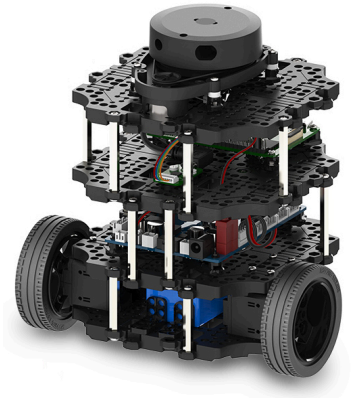


Fig 4. Turtlebot3 Burger Model

The setup process to achieve multi-robot collaboration involves several stages. The first step is hardware assembly, following the official TurtleBot3 e-manual. Subsequently, the host PC must be configured with the ROS 2 framework and the required TurtleBot3 software packages, as outlined in Chapter 3.1 of the e-manual. On the robot side, each Raspberry Pi must also be configured with ROS 2 and the necessary hardware interfaces, following Chapters 3.2 and 3.3.

Once the basic Turtlebot3 setup is completed, multi-robot collaboration can be enabled by following the guidelines provided in Chapter 10.11.4 of the official TurtleBot3 e-manual. This configuration outlines the necessary modifications for deploying multiple robots within a shared system, including the use of unique namespaces to isolate each robot's topics and nodes, as illustrated in Fig. 4.

[17] ROBOTIS. (2026). *TurtleBot3 Features – Specifications*. https://emanual.robotis.com/docs/en/platform/turtlebot3/features/#specifications

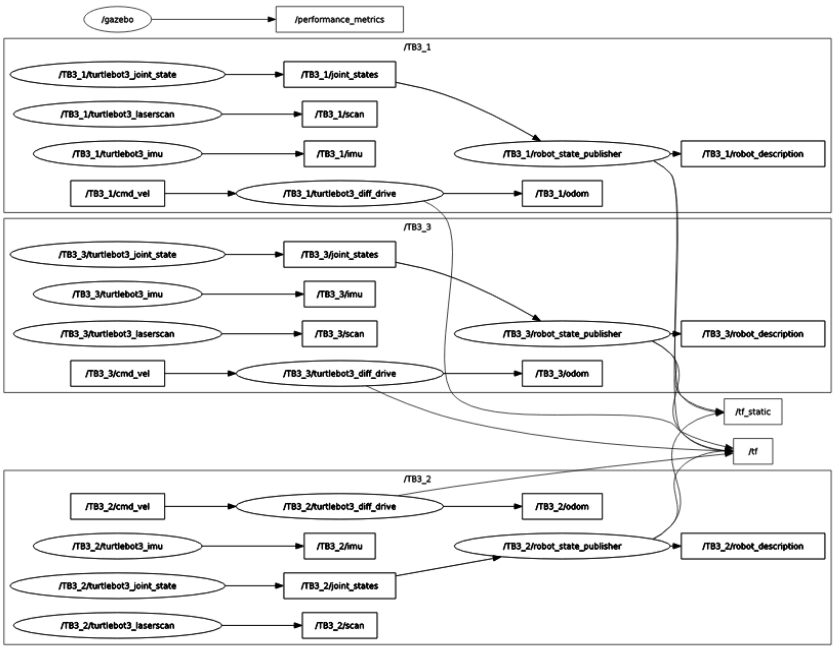


Fig 5. Namespaces in Turtlebot3 Multi-Robot Collaboration

In this study, a total of four TurtleBot3 units are utilized. Each robot operates within a unique namespace to prevent conflicts in communication topics and node identifiers. Specifically, the robots are assigned the namespaces *tb3_1*, *tb3_2*, *tb3_3*, and *tb3_4*, respectively.

Despite the use of namespaces, all TurtleBot3 units continue to publish to the shared *tf* and *tf_static* topics, in accordance with the configuration specified in the e-manual. This shared transformation framework enables consistent frame management across all robots.

Other than the shared *tf* and *tf_static* topics, it is also crucial to note that all robots operate within the same ROS 2 domain ID. This unified domain serves as the foundation for multi-robot collaboration, enabling higher-level applications such as collaborative SLAM and coordinated task execution.

3.2 Cartographer Setup

After configuring the TurtleBot3 platforms, modifications were made to the default Cartographer setup provided in the *turtlebot3* official repository, particularly in the *turtlebot3_cartographer* directory. The original implementation is designed for a single robot and does not natively support multi-robot deployments with separate namespaces.

To enable concurrent operation of multiple TurtleBot3 units, the Cartographer configuration was extended to incorporate robot-specific namespaces. This required updates to several components, including the Lua configuration files, launch files, and RViz configuration. By introducing namespaces into these components, each robot can independently run localization and mapping while avoiding topic and frame conflicts within the shared ROS 2 network.

The modified implementations for multi-robot Cartographer setups are available in the turtleswarm repository, specifically under the directories *tb3_1_cartographer*, *tb3_2_cartographer*,

tb3_3_cartographer, and *tb3_4_cartographer*. Each directory corresponds to a specific robot instance with its respective namespace configuration.

3.3 Nav2 Setup

The Navigation 2 (Nav2) stack is an open-source ROS 2 navigation framework that uses a behaviour tree-based architecture to orchestrate a set of modular servers[18]. A global planner computes a collision-free path using a costmap derived from the SLAM-generated map, while a local controller tracks the path in real time by issuing velocity commands based on a continuously updated local costmap. Nav2 is deployed here to enable autonomous navigation for each TurtleBot3 within a multi-robot environment. For our configuration of Nav2 for multi-robot operation[19], each robot operates under a unique namespace (e.g., *tb3_1*, *tb3_2*) to isolate topics, frames, and services across multiple agents.

To ensure compatibility between Nav2 and the SLAM system, parameter files are dynamically rewritten at launch time. This includes prefixing all frame IDs (e.g., map, odom, base_footprint) with the corresponding robot namespace, allowing each robot to maintain an independent coordinate frame within the shared system.

A key challenge arises from the fact that the TurtleBot3 hardware and SLAM components publish transformation data (*tf* and *tf_static*) to the global (*root*) namespace, whereas Nav2 expects these topics to be available within the robot-specific namespace (e.g., *tb3_2/tf*). To resolve this mismatch, relay nodes are introduced.

Specifically:

1. A *tf_relay* node forwards transformation data from tf to *{namespace}/tf*
2. A *tf_static_relay* node forwards static transformations from *tf_static* to *{namespace}/tf_static*

This relaying mechanism ensures that Nav2 can correctly receive and interpret transformation data required for localization, planning, and control.

In addition, SLAM is performed using *slam_toolbox*, which is intentionally launched without a namespace to avoid incorrect topic resolution (e.g., duplicated namespace prefixes). As a result, SLAM publishes outputs such as the occupancy grid map (*map*) and transformations (*tf*) in the root namespace. To integrate these outputs with Nav2, additional relay nodes are used:

1. A map relay forwards *map* to *{namespace}/map*
2. A map updates relay forwards incremental updates accordingly

These relays enable Nav2's global costmap to subscribe to the correct map topic and maintain real-time updates during navigation.

[18] *Nav2—Nav2 1.0.0 documentation*. (n.d.). Retrieved April 18, 2026, from https://docs.nav2.org/
[19] *GitHub—Nishuy52/tb3_2_navigation2 at auto_nav2*. (n.d.). GitHub. Retrieved April 18, 2026, from https://github.com/Nishuy52/tb3_2_navigation2/tree/auto_nav2

3.4 Autonomous Navigation Setup

The autonomous navigation (AutoNav) component implements frontier-based exploration, enabling each TurtleBot3 to map an unknown environment without human intervention. The system operates in two sequential phases. In the bootstrap phase, the robot executes a short open-square trajectory via Nav2's *NavigateToPose* action to seed SLAM with an initial map before exploration begins. Once a sufficient map footprint exists, the system transitions to the exploration phase, in which it iteratively detects, clusters, and navigates toward frontier cells which are defined as free cells adjacent to unexplored space, until the fraction of unknown cells in the occupancy grid falls below a configurable threshold (default 5%).

The AutoNav node subscribes to the namespaced occupancy grid (e.g., *tb3_2/map*) and uses a TF buffer to resolve the robot's current pose in the map frame by looking up the transform from *{namespace}/map* to *{namespace}/base_footprint*. Navigation goals are issued as *PoseStamped* messages referencing the *{namespace}/map* frame and dispatched asynchronously to the Nav2 *NavigateToPose* action server at */{namespace}/navigate_to_pose*.

Frontier detection is performed directly on the occupancy grid using a 4-connected scan for free cells neighbouring unknown cells. Detected frontier cells are greedily clustered by proximity (radius 0.5 m), and clusters below a minimum size of three cells are discarded as noise. The nearest non-blacklisted cluster centroid is selected as the next navigation goal, shifted slightly toward the robot to ensure it lies within free space.

To handle cases where the robot becomes stuck, a dual blacklisting mechanism is employed: a frontier is blacklisted after a navigation abort or after being re-selected a configurable number of consecutive times. When the entire frontier set is blacklisted, the blacklist is cleared and exploration resumes, preventing permanent deadlock.

As with Nav2, all AutoNav topics and TF lookups operate within the robot-specific namespace. The node relies on the same relay infrastructure described in Section 3.3 of the map relay, map-updates relay, tf relay, and tf_static relay to receive the SLAM outputs published in the root namespace and expose them under the correct namespaced topics that both Nav2 and AutoNav expect.

3.5 Experimental Overview

Following the completion of the system setup, latency measurements were conducted to evaluate communication performance. The experimental framework consists of a laptop acting as the publisher node and multiple TurtleBot3 platforms acting as subscriber nodes. The system was implemented using ROS 2 Humble.

End-to-end latency was measured using a timestamp-based approach implemented through custom ROS 2 nodes. The publisher node transmits velocity commands in the form of *TwistStamped* messages, where each message includes a timestamp corresponding to its transmission time.

During each run, the publisher sends 10 consecutive timestamped messages on a dedicated ROS 2 topic. On the robot side, each Raspberry Pi receives the messages and records the corresponding

reception time. The latency for each message is then computed as the difference between the embedded transmission timestamp and the local reception time at the subscriber.

To ensure reliability and statistical robustness, each experimental configuration was repeated five times, with 10 latency samples collected per run. This results in a total of 50 latency measurements per configuration. Repeated sampling helps mitigate the effects of transient variations in wireless communication and system performance, thereby providing a more representative evaluation of latency behavior.

As the experiment involves multiple devices, specifically the TurtleBot3 units and the laptop, clock synchronization is essential for accurate latency measurement. The Chrony package was used to synchronize system clocks across devices. The laptop was configured as the reference time source, while each Raspberry Pi synchronized its clock accordingly. This step is critical, as latency calculations rely on timestamps generated on different devices; without proper synchronization, clock drift would introduce significant measurement errors.

This study comprises two independent experiments, each designed to isolate specific factors affecting communication latency:

1. Experiment 1: Impact of network infrastructure under increasing system load
2. Experiment 2: Impact of DDS middleware under controlled network conditions and increasing system load

Each experiment employs its own set of control variables to ensure fair and meaningful comparisons. The experimental design is motivated by a layered systems perspective inspired by the OSI model, where system performance is influenced by multiple abstraction layers ranging from underlying network infrastructure to higher-level middleware behavior.

In this context, the study adopts a bottom-up evaluation strategy, beginning with lower-level communication layers before progressing to higher-level components.

In Experiment 1, the focus is placed on the network infrastructure, which forms the foundation of all inter-process communication. Since latency introduced at this level propagates upward and affects all higher-level operations, it is necessary to first characterize and optimize the network under increasing system load. This establishes a reliable baseline and identifies the most stable network configuration for subsequent analysis.

In Experiment 2, the network configuration identified from Experiment 1 is fixed to eliminate variability from lower layers. This enables the study to isolate the impact of DDS middleware on communication latency. Consequently, any observed performance differences can be attributed to middleware-level design choices rather than underlying network conditions.

3.4.1 Experiment 1: Network Infrastructure vs System Load

This experiment evaluates how different student-accessible network infrastructures influence communication latency as system load increases.

In the context of the NUS IDP, two common types of network infrastructure are typically used: a mobile hotspot and a Huawei router provided to students. In this study, the mobile hotspot is specifically provided by a Samsung Galaxy A25 5G operating on the 2.4 GHz band, while the router used is the Huawei Mobile WiFi Pro 2 (E5885Ls-93a). The mobile hotspot represents a practical and widely used setup where students rely on personal devices for connectivity, while the Huawei router reflects the standard networking equipment commonly provided to students within the IDP environment.

To simulate increasing system load, this experiment utilizes the Cartographer-based SLAM system described in Chapter 3.2. The workload is scaled by varying the number of TurtleBot3 units concurrently running Cartographer, thereby increasing the volume of communication and computational demand within the system.

The experimental variables are defined as follows.

Variables Type	**Elements**
Control variables	● DDS middleware (Fast DDS) ● Experiment physical location (NUS E2A Studio 3) ● Hardware configuration ● Message type and measurement procedure
Independent variables	● Network type: 2.4 GHz mobile hotspot (Samsung Galaxy A25 5G) or Huawei router ● System load: number of Turtlebot3 (up to 4) units running Cartographer
Dependent variable	End-to-end latency

Table 1. Experimental variables in Experiment 1

This experiment captures how network limitations interact with increasing SLAM workload, particularly in conditions representative of typical student setups. Moreover, this experiment also serves as the basis for Experiment 2 by determining the appropriate number of TurtleBot3 units and network configuration for further optimization of end-to-end latency.

3.4.2 Experiment 2: DDS Middleware vs Workload Intensity

Building upon the first experiment, which examines the impact of network infrastructure, this experiment focuses on the influence of DDS middleware under increased system workload. In addition to the Cartographer-based SLAM system, a more computationally intensive workload is introduced in the form of autonomous navigation, incorporating Nav2 alongside custom autonomous control algorithms. This setup reflects a more realistic and demanding multi-robot application scenario.

The objective of this experiment is to evaluate how different DDS middleware implementations perform as system workload increases. In particular, the default ROS 2 middleware, Fast DDS, is compared against an alternative implementation, Cyclone DDS, to determine whether differences in middleware design lead to measurable variations in communication latency.

The experimental variables are defined as follows.

Variables Type	**Elements**
Control variables	● Network configuration (Samsung Galaxy A25 5G 2.4 GHz mobile hotspot) ● Experiment physical location (NUS E2A Studio 3) ● Hardware configuration ● Message type and measurement procedure
Independent variables	● DDS middleware: Fast DDS or Cyclone DDS ● System load: number of Turtlebot3 (up to 2) running Nav2 with varying workload intensity
Dependent variable	End-to-end latency

Table 2. Experimental variables in Experiment 2

The mobile hotspot was fixed as the network condition in this experiment to isolate the effect of DDS middleware. Preliminary observations indicated that the Huawei router exhibited lower and less stable performance, which could confound the comparison between middleware implementations. By maintaining a consistent network condition, this experiment ensures that observed latency differences are attributable to DDS behavior rather than network variability.

4. Results and Discussion

4.1 Experiment 1: Network Infrastructure vs System Load

4.1.1 Results

Following the procedure described in Chapter 3.4.1, end-to-end latency data were collected and analyzed for Experiment 1.

The results are summarized in Fig. 5, which illustrates the average end-to-end latency under two network configurations—a 2.4 GHz mobile hotspot and a Huawei router—as the number of TurtleBot3 units increases from one to four.

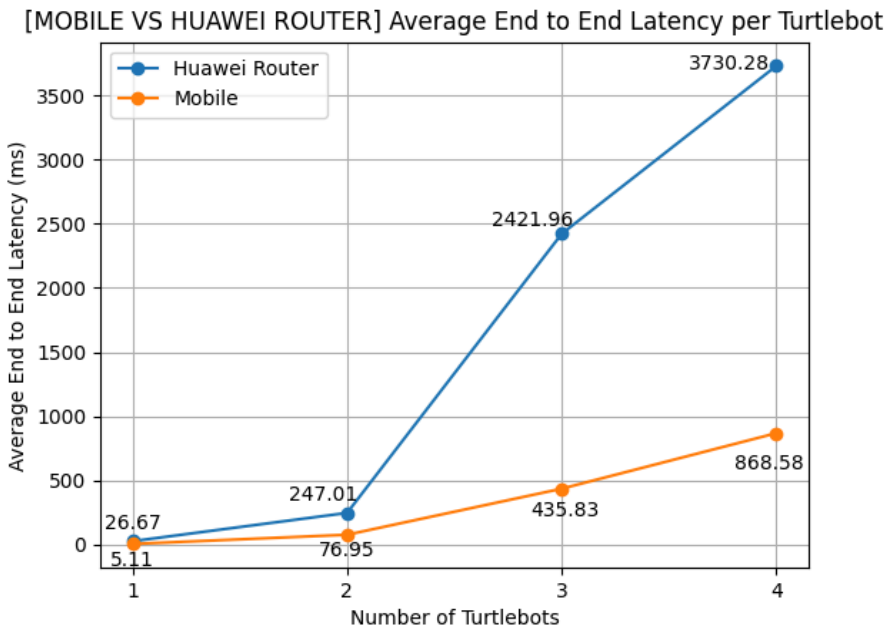


Fig 6. Mobile hotspot versus Huawei router average latency

In general, latency increases with the number of robots for both network configurations, reflecting the effect of increasing system load. However, the rate of increase differs significantly between the two networks.

With a single TurtleBot3, both configurations exhibit low latency, with the mobile hotspot performing slightly better than the Huawei router. As the system scales to two robots, latency increases in both cases, although the increase is more pronounced for the Huawei router.

At three robots, the disparity becomes more evident: latency under the Huawei router rises sharply, while the mobile hotspot continues to show a more gradual increase. This trend persists at four robots, where the Huawei router exhibits a steep escalation in latency, reaching the highest observed values, whereas the mobile hotspot maintains a comparatively moderate growth.

In addition to latency, message reliability was also evaluated. As system load increases, message drops are observed under the Huawei router configuration. The extent of this effect is illustrated in Fig. 6, which presents the number of dropped messages for both network setups.

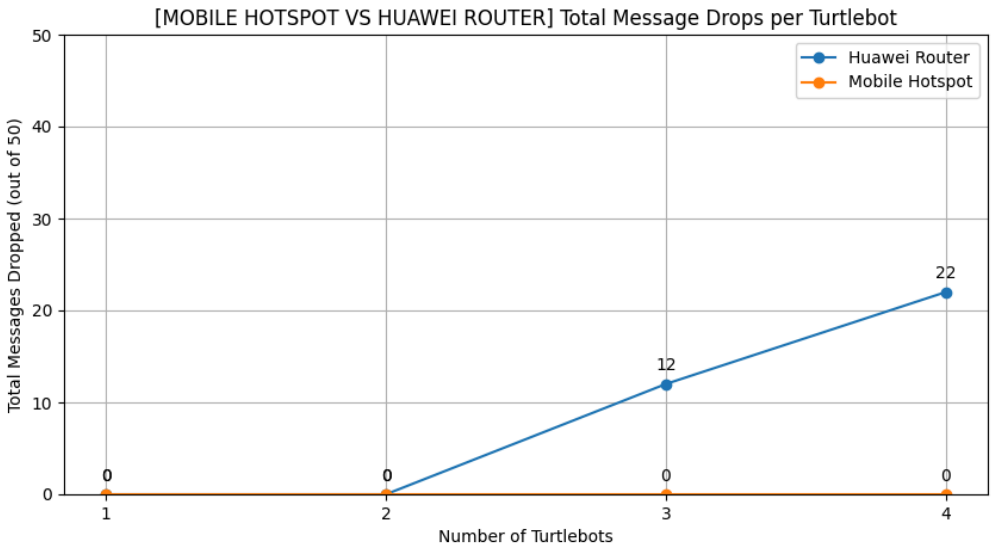


Fig 7. Mobile hotspot versus Huawei router message drops

Overall, the Huawei router demonstrates a significantly steeper increase in latency as system load rises, accompanied by message loss when the workload exceeds its handling capacity. In contrast, the mobile hotspot maintains lower latency and more stable performance across all tested configurations.

4.1.2 Discussion

The results indicate that network infrastructure plays a significant role in determining system performance, particularly under increasing workload. The Huawei router exhibits a steeper increase in latency and begins to experience message loss as the number of robots increases, suggesting that it is less capable of handling high communication demand.

In contrast, the mobile hotspot demonstrates lower latency and more stable performance across all configurations, indicating better handling of increased communication load in this experimental setup.

Based on these observations, a two-robot configuration was selected for Experiment 2. At three robots, the latency under the Huawei router becomes significantly high, indicating system saturation. Therefore, to ensure a controlled and reliable experimental environment, Experiment 2 focuses on two robots while increasing workload through Nav2-based autonomous navigation.

Furthermore, the mobile hotspot configuration was chosen for Experiment 2, as it provides the most stable and efficient network performance among the tested setups.

4.2 Experiment 2: DDS Middleware vs Workload Intensity

4.2.1 Results

Following the procedure described in Chapter 3.4.2, end-to-end latency data were collected and analyzed for Experiment 2.

The latency results comparing Fast DDS and Cyclone DDS across different workload scenarios are presented in Fig. 7.

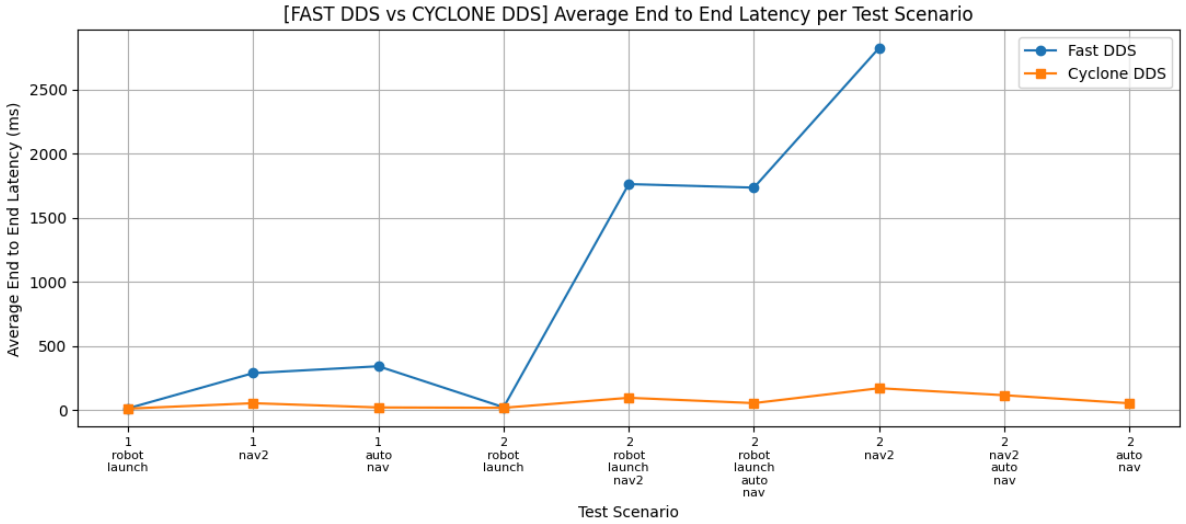


Fig 8. Fast DDS versus Cyclone DDS average latency

In simple scenarios, such as a single robot launch, both middleware implementations exhibit low latency. As the workload increases to moderate levels—such as a single robot running Nav2 or executing autonomous navigation—Fast DDS begins to show higher latency compared to Cyclone DDS.

In multi-robot scenarios, latency increases for both middleware implementations due to increased communication and computational demands. However, the increase is significantly more pronounced for Fast DDS, particularly when multiple robots are running Nav2 simultaneously.

In contrast, Cyclone DDS maintains relatively lower latency across all configurations, with more gradual increases as workload intensifies. Consequently, the performance gap between the two middleware implementations becomes more evident at higher workload levels.

The reliability of communication was further evaluated through message drop analysis, as shown in Fig. 8.

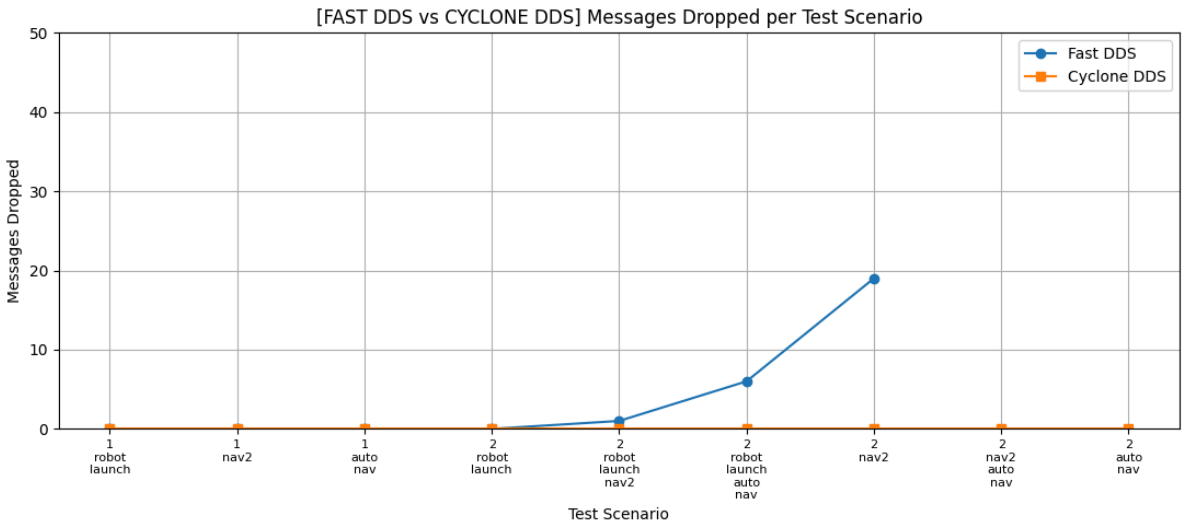


Fig 9. Fast DDS versus Cyclone DDS message drops

Across all tested scenarios, Cyclone DDS consistently records zero message drops, indicating stable communication performance. Fast DDS, on the other hand, exhibits no message loss under lighter workloads, such as single robot launch and basic navigation scenarios. However, as workload increases, particularly in multi-robot configurations involving Nav2 or autonomous navigation, message drops begin to occur.

The number of dropped messages increases further under more demanding configurations, reaching the highest values in the most intensive scenarios. This suggests that Fast DDS experiences reduced reliability under high system load, whereas Cyclone DDS remains robust across all tested conditions.

4.2.2 Discussion

The observed results are consistent with the differences in DDS implementation discussed in Chapter 2.3. In Fast DDS, the default synchronous publication mode executes message transmission within the application thread, resulting in a blocking behaviour. Under low workload conditions, this does not significantly impact performance, which explains the similarly low latency observed in simple scenarios. However, as workload increases, particularly in Nav2-based multi-robot systems, the blocking nature of synchronous execution introduces additional delay, leading to higher latency and eventual message drops when the system becomes saturated. As such, we observe the complete degradation of the Nav2 system, with enough message drops that slam_toolbox can no longer generate maps as seen in Fig 9.

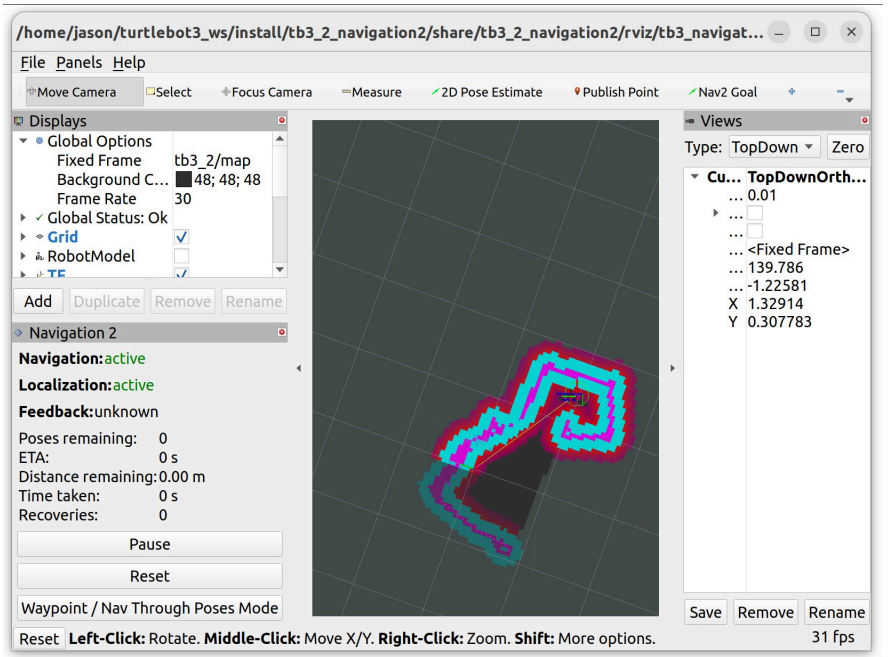


Fig 10: RViz View after 5 minutes when running Nav2 stack with another robot bringup

In contrast, Cyclone DDS employs a multi-threaded processing pipeline, where message handling tasks are distributed across multiple threads. This design enables concurrent processing of communication tasks, reducing contention and preventing the application thread from being blocked. As a result, Cyclone DDS is able to sustain higher throughput and maintain lower latency even under high-frequency workloads. This also explains the absence of message drops across all tested scenarios, as the system is better able to keep up with incoming message rates.

Therefore, the experimental results support the literature: Cyclone DDS achieves more stable and scalable performance under increasing workload, while Fast DDS is more susceptible to latency degradation and reliability issues due to its synchronous execution model.

To further illustrate the system performance, the final generated map is presented. The map corresponds to the Cyclone DDS configuration under a two-robot autonomous navigation scenario, demonstrating stable operation under moderate workload conditions.

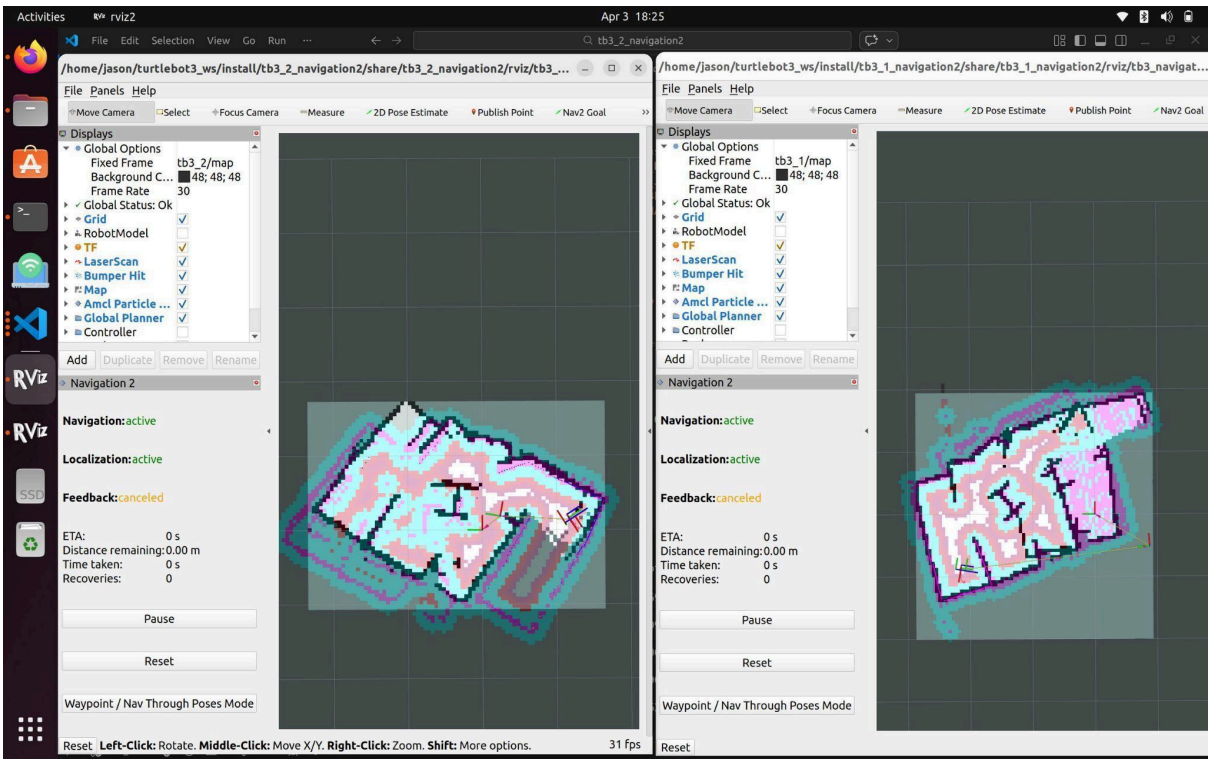


Fig 11. Two-robot Autonomous Navigation Maps (Nav2) in Cyclone DDS

5. Future Improvements

5.1 Migration to ROS 2 Jazzy

One potential direction for extending this work is the migration from ROS 2 Humble to Jazzy Jalisco. While Humble is a Long-Term Support (LTS) release and remains widely adopted in educational settings, newer distributions such as Jazzy incorporate improvements in system stability, middleware performance, and developer tooling[20].

Recent ROS 2 releases have focused on enhancing communication reliability, executor efficiency, and DDS integration, all of which are directly relevant to latency-sensitive multi-robot systems. Additionally, a growing portion of the robotics community is building and maintaining packages on newer distributions, making Jazzy increasingly relevant for future-proof system design. Migrating to Jazzy would therefore allow this study to evaluate whether improvements at the middleware and runtime levels translate into measurable reductions in end-to-end latency under similar experimental conditions.

5.2 Exploration of Zenoh-Based Communication

Another promising area of exploration is the adoption of Zenoh as an alternative to conventional DDS-based communication in ROS 2. Zenoh is a data-centric communication protocol designed to unify data in motion, data at rest, and computation, while providing efficient routing over heterogeneous networks.

Unlike traditional DDS implementations, Zenoh supports flexible communication patterns, including peer-to-peer, brokered, and hybrid architectures. This allows better isolation of communication flows and can reduce unnecessary network congestion, particularly in distributed robotic systems operating over constrained wireless networks. Moreover, Zenoh has been explored as a backend for ROS 2 through the Zenoh-DDS bridge, demonstrating its capability to provide scalable and robust communication with lower overhead in certain scenarios[21][22].

Investigating Zenoh within the context of this study could provide insights into whether alternative communication paradigms can mitigate latency bottlenecks observed in DDS-based systems, especially as the number of robots and communication load increases.

5.3 Multi-Agent SLAM and Collaborative Mapping

This study focuses primarily on communication latency in multi-robot systems, with each TurtleBot3 operating on an independent map. A natural extension of this work is to explore collaborative mapping through multi-agent SLAM[23].

---

[20] Open Robotics. (2024). *Jazzy Jalisco (jazzy) — ROS 2 Documentation*. https://docs.ros.org/en/jazzy/Releases/Release-Jazzy-Jalisco.html

[21] Liao, Z., et al. (2024). Comparison of Middlewares in Edge-to-Edge and Edge-to-Cloud Communication for Distributed ROS 2 Systems. *arXiv:2309.07496v4*. https://arxiv.org/abs/2309.07496

[22] Bozzini, M., et al. (2025). Performance Comparison of ROS 2 Middlewares for Multi-robot Mesh Networks in Planetary Exploration. *Journal of Intelligent & Robotic Systems*, 111, 18. https://link.springer.com/article/10.1007/s10846-024-02211-2

[23] Lajoie, P. Y., & Beltrame, G. (2023). Swarm-SLAM: Sparse Decentralized Collaborative Simultaneous Localization and Mapping Framework for Multi-Robot Systems. *IEEE Robotics and Automation Letters*, 9, 475–482.

In multi-agent SLAM, multiple robots simultaneously build and share a unified map of the environment. This requires frequent exchange of sensor data, pose estimates, and map updates, significantly increasing communication demands compared to independent operation. While this introduces additional complexity, it also enables more efficient exploration, redundancy, and robustness in large-scale environments.

With a well-characterized and optimized communication infrastructure such as those evaluated in this study, future work can investigate how latency impacts the consistency and convergence of shared maps. This would extend the scope of the research from communication performance analysis to system-level behavior, where network efficiency directly influences mapping accuracy and coordination effectiveness.

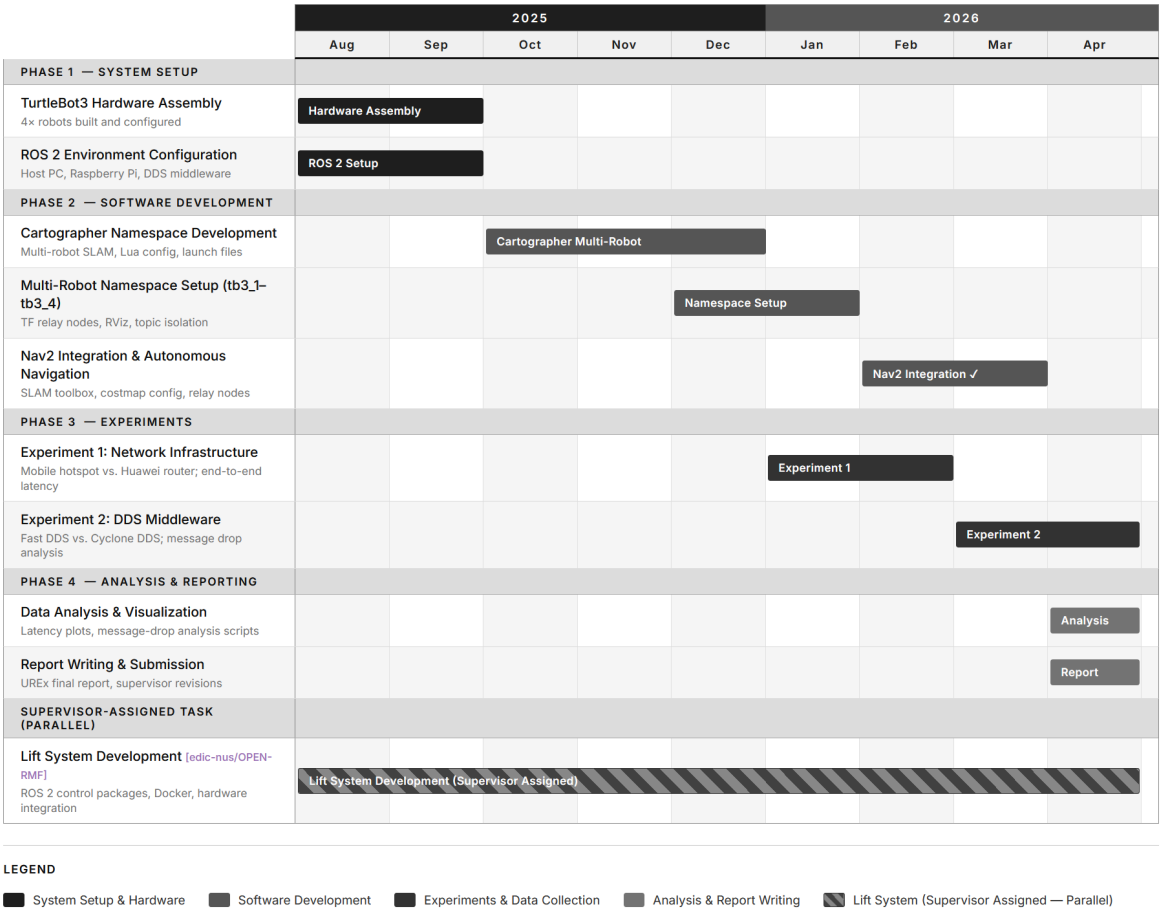
6. Conclusion

This study examined end-to-end communication latency in multi-robot ROS 2 systems built on TurtleBot3 platforms, with the goal of identifying practical infrastructure configurations for educational settings such as the NUS IDP. Two controlled experiments were conducted: one isolating the effect of network infrastructure (mobile hotspot versus Huawei router) and one comparing DDS middleware implementations (Fast DDS versus Cyclone DDS) under increasing system workload.

Experiment 1 showed that the Samsung Galaxy A25 5G mobile hotspot consistently delivered lower latency and fewer message drops than the Huawei router as the number of active robots increased. Experiment 2 demonstrated that Cyclone DDS significantly outperforms Fast DDS at higher workloads, owing to its multi-threaded architecture, whereas Fast DDS's synchronous execution model leads to latency degradation and message loss under heavier loads.

Based on these results, a two-robot configuration using a mobile hotspot and Cyclone DDS is recommended as a practical baseline for IDP multi-robot deployments. Future work may build on this foundation by migrating to ROS 2 Jazzy, exploring Zenoh-based communication, and extending the system to multi-agent SLAM to further improve scalability in constrained environments.

Appendix 1: Project Timeline



Appendix 2: Bill of Materials

S/N	Item	Amount (SGD)
1	ROSCon Full Conference Registration	1296.06
2	ROSCon Workshop: Hands-On Aerial Robotics Using PX4	64.80
3	ROSCon Workshop: How to implement a full ROS 2 application: a tic-tac-toe robot	64.80
4	ROSCon Workshop: ros2_control: Fun With Robot	64.80
5	SIMBA SIM Card Mobile Number 89179761 SuperRoaming	10.00
Total		1500.47